worldskills Asia Taipei2025

Test Project

Cloud Computing

Day1

WorldSkills Asia acknowledges WorldSkills International's copyright and intellectual property rights.
This document has been adapted for WorldSkills Asia with express permission from WorldSkills International.

The release date is November 27th, 2025(C1). Please do not disclose this document before that date.

Content

The Test Project consists of the following:

1 Task Description 3
2 Background 3
3 Stages 4
4 Configuration details 6
5 Reference 10

Task Description

GreenTree provides REST APIs for planting trees in cyberspace. In this challenge, you will work in AWS cloud environment. As a cloud engineer, your task is to design and implement the infrastructure so that the service can operate reliably and efficiently.

You will build the infrastructure using AWS services and configure networking, container and database resources. In addition, you must deploy the provided binaries and develop additional APIs to serve the GreenTree applications. You are also responsible for system operations, ensuring high availability, performance tuning, security, disaster recovery, scalability and observability.

Furthermore, the task involves not only setting up AWS services but also configuring open-source solutions to build a CI/CD pipeline and a Single Sign-On (SSO) platform.

Background

GreenTree is a cloud-based service. As the customer base has grown, the company has decided to expand the system to a multi-region architecture. Although the service will operate across multiple regions, it aims to provide customers with the same experience regardless of where they connect from. The system may need a data synchronization strategy.

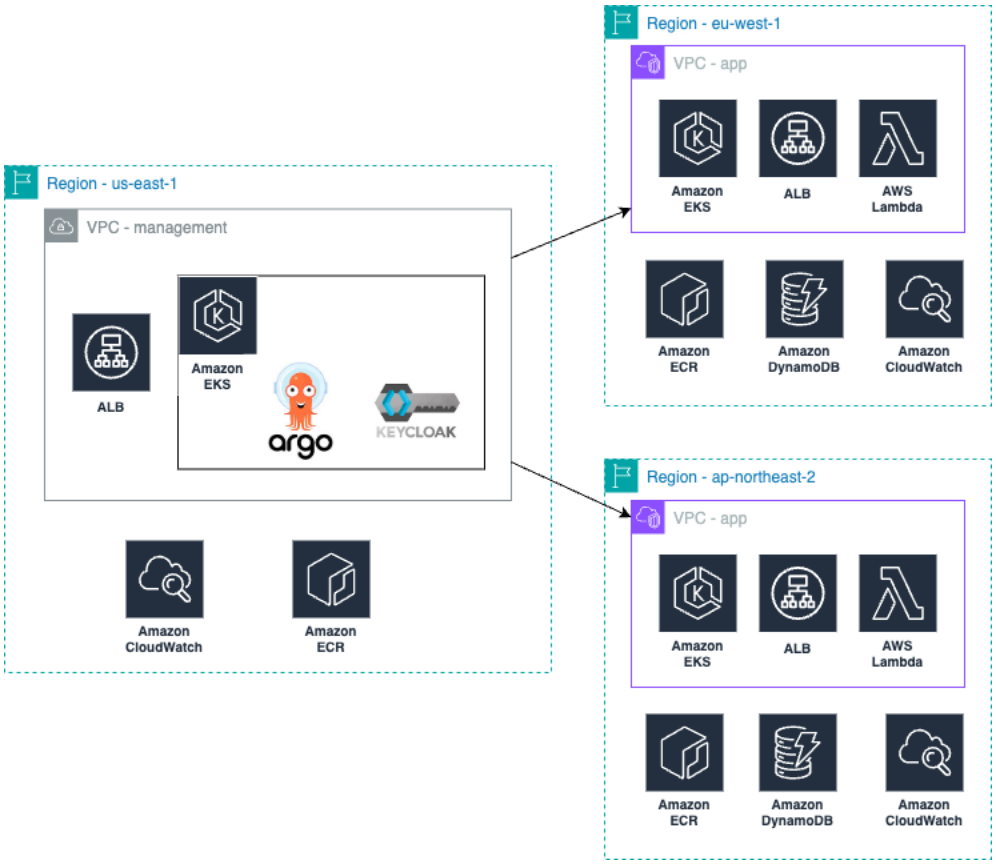
A strategy is required to efficiently manage multiple servers when deploying the same application across multiple regions. For centralized management, you will first create a Management VPC and deploy tools such as ArgoCD. After that, you will provision App VPCs and deploy the GreenTree applications.

A former employee developed the API used to check the status of trees. However, the latest version of the code was never committed, and the API contains issues. Therefore, it has been decided to redevelop this API and deploy it using AWS Lambda. You are required to develop and deploy the Lambda function accordingly.

Developers have reported inconvenience due to having separate login systems across various tools. To improve the login experience, you will set up a Single Sign-On (SSO) system. The goal is to enable users to log in with the same account across multiple tools such as AWS Management Console and ArgoCD.

Security configurations must be implemented to ensure that different users and applications have appropriate permissions, and observability should be considered for effective system operation.

Overall architecture



Stages

Stage 1: Networking

You need to configure a Management VPC and multiple App VPCs. Each VPC must include public subnets that can communicate directly with the internet and private subnets that can access the internet through a NAT gateway. The Management VPC is located in the us-east-1 region, and App VPCs are located in the ap-northeast-2 and eu-west-1 regions. The Management VPC is connected to the App VPC via VPC Peering. In addition, the servers in the App VPCs must access DynamoDB through a VPC Endpoint.

After completing this step, the VPCs will be set up, communication between VPCs will be possible, and access to DynamoDB via the VPC Endpoint will be available.

Stage 2: Management services

An EKS cluster should be created in the Management VPC. ArgoCD and Keycloak must be installed on the Management EKS to support container deployment and to set up the Single Sign-On (SSO) system. ArgoCD will be used to deploy container-based applications, including Keycloak on the Management EKS, and the Plant and Grow applications on the App EKS. Keycloak will use an Aurora PostgreSQL database and serve as the SSO platform for AWS login and ArgoCD access.

After completing this step, the Management EKS will be set up, and ArgoCD and Keycloak will be running on the cluster.

Stage 3: Application resources

In the EU and AP App VPCs, you are required to configure EKS, ECR, DynamoDB and application deployment. First, create the App EKS clusters and worker nodes using a convenient method. Once the clusters are ready, integrate them with ArgoCD running on the Management EKS. After the integration, you can create ArgoCD Applications and deploy resources through ArgoCD.

Next, deploy DynamoDB-ACK in each App EKS cluster and create DynamoDB tables using ArgoCD. After creating the tables, you must manually build container images for the Grow and Plant applications, upload them to ECR, and prepare Helm charts for deployment. Also, deploy Grow and Plant resources to each App EKS cluster. Additionally, configure Karpenter in the App EKS clusters to enable pod autoscaling.

After completing this step, ECR repositories, Helm charts, App EKS clusters, and DynamoDB tables will be set up. DynamoDB-ACK, the green-tree table, Grow, and Plant will be created as ArgoCD Applications, and the corresponding Kubernetes objects are deployed to each App EKS. As a result, you must provide a single endpoint for invoking the Grow and Plant APIs.

Stage 4: Lambda function implement

You need to develop the Status API that retrieves the status of trees from DynamoDB. Implement the function in your preferred programming language and deploy it as a Lambda function in the EU and AP App VPCs. The single endpoint must allow not only the Plant and Grow APIs but also the Status API to be invoked.

After completing this step, the Lambda function named status-api will be deployed in the App VPCs, and the Status API must be accessible from the single endpoint.

Stage 5: Single Sign-On

Users in Keycloak must be able to log in to both the AWS Management Console and ArgoCD. Create two users in Keycloak, admin001 and dev001, and configure Keycloak and Service Providers to perform authentication and authorization based on the group information of Keycloak users. In AWS, Access must be granted through IAM roles, and in ArgoCD, permissions must be enforced according to RBAC definitions.

After completing this step, users in Keycloak will be able to log in to both ArgoCD and the AWS Management Console.

Stage 6: Observability

Logs from the EKS clusters, VPC Flow Logs, application logs, and Lambda function logs must be stored in CloudWatch Logs. In addition, enable database performance metrics. A CloudWatch Dashboard should be

created to allow easy monitoring of the overall system status.

After completing this step, EKS, VPC Flow Logs, application, and Lambda logs will be stored, and a dashboard will be available to monitor the overall system health.

Configuration details

VPC

- For high availability, each VPC must have at least two subnets.
- Subnets must have a Name tag with either "public-subnet-" or "private-subnet-" as its prefix.
- The Management VPC and the App VPCs are connected via VPC peering.
- Servers in App VPCs must use a VPC Endpoint to connect DynamoDB in order to enhance security and reduce costs.
- The DynamoDB VPC endpoint should be configured in the route table rather than by using the ENI method.
- Ensure that there are no overly permissive SSH ingress rules (e.g., 0.0.0.0/0) in any Security Group.

EKS

- All EKS clusters should be created as private clusters.
- Create the Management EKS and App EKS clusters using your preferred method, such as Terraform or eksctl.
- Deploy ArgoCD on the Management EKS with Helm, Kustomize or another method.
- Create Keycloak as an Application in ArgoCD and deploy it on the Management EKS through ArgoCD.
- Create Applications named DynamoDB-ACK-AP and DynamoDB-ACK-EU in ArgoCD and deploy them on the App EKS clusters through ArgoCD.
- Create Applications named GreenTree-table-AP and GreenTree-table-EU in ArgoCD and provision the green-tree DynamoDB tables.
- Create Applications named Karpenter-AP and Karpenter-EU in ArgoCD and deploy the resources to each App EKS cluster using ArgoCD.
- Create Applications named Plant-AP and Plant-EU in ArgoCD and deploy the resources to each App EKS cluster using ArgoCD.
- Create Applications named Grow-AP and Grow-EU in ArgoCD and deploy the resources to each App EKS cluster using ArgoCD.
- Plant and Grow resources should be deployed in the green-tree namespace.
- Resource requests and limits must be set for the Plant and Grow Pods.
- Configure an HPA-based auto-scaling policy for Pods in the green-tree namespace.
- Use t3.medium EC2 instance. Other types of EC2 or Fargate are not allowed.

ECR

- Create two ECR repositories for the GreenTree applications; Plant and Grow.
- Build and upload at least one container image.
- The Plant and Grow repositories must be replicated across two regions: ap-northeast-2 (primary) and eu-west-1 (replica).
- All images should be encrypted with a Customer Managed Key (CMK).
- If an image with the same tag already exists, the upload must be denied.
- Plant and Grow Pods in AP must use the ECR repositories in AP.
- Plant and Grow Pods in EU must use the ECR repositories in EU.
- Configure a lifecycle policy to delete images older than 14 days from the upload date.

ArgoCD

- Install ArgoCD on the Management EKS cluster with your preferred method.
- The ArgoCD address is https://argocd.<competitor's base domain>
- Ensure that the following applications are visible on the ArgoCD web console: DynamoDB-ACK-EU, DynamoDB-ACK-AP, GreenTree-table-EU, GreenTree-table-AP, Keycloak, Karpenter-EU, Karpenter-AP, Plant-EU, Plant-AP, Grow-EU and Grow-AP.
- Ensure that you can access its web console from your desktop browser.
- You need a Helm chart repository for Plant and Grow, and you may use S3 or ECR instead of Github.

DynamoDB

- Create DynamoDB tables named green-tree in both the ap-northeast-2 and eu-west-1 regions using DynamoDB ACK and ArgoCD.
- The green-tree tables must be created as a Global Table to enable automatic replication between the ap-northeast-2 and eu-west-1 regions.
- The green-tree table must be encrypted with a CMK and configured to use on-demand mode.
- Enable backup retention for more than 7 days, and ensure that at least one backup exists.
- The Plant, Grow and Status applications use the green-tree table to write the following data fields: treeId, treeName, sunlight and water.
- Each application must have permissions to access only the green-tree table through its IAM role.

Aurora DB

- Create an Aurora PostgreSQL database named keycloak.
- Use the instance class db.t3.medium.
- Set the DB identifier to keycloak.

- Enable backup retention for more than 7 days, and ensure that at least one backup exists.
- Change the database port to 10540.
- The Keycloak Pod uses the keycloak database.

Keycloak

- Deploy Keycloak on the Management EKS cluster.
- The system uses the Aurora PostgreSQL database.
- Ensure that you can access its web console from your desktop browser.
- The Keycloak address is https://keycloak.<competitor's base domain>.
- Create two groups named admin and developer.
- Create the admin001 user and add it as a member of the admin group.
- Create the dev001 user and add it as a member of the developer group.
- Set the same password for all users and write it into the box.

_____

- AWS SSO Login with SAML Integration
  - Use "Identity providers" in the IAM page instead of IAM Identity Center.
  - Create two AWS IAM roles named admin and developer, and attach the AdministratorAccess policy to the admin role and the ReadonlyAccess policy to the developer role.
  - The admin001 Keycloak user must be able to assume the admin IAM role after logging in through Keycloak.
  - The dev001 Keycloak user must be able to assume the developer IAM role after logging in through Keycloak.
  - The login URL is: <keycloak DNS>/realms/master/protocol/saml/clients/asiaskills
- ArgoCD SSO integration.
  - The admin001 user has all permissions.
  - The dev001 user must only be able to view the Plant and Grow Applications, including their associated resources.

Lambda

- Implement the GET /status API according to the specification, using any programming language.
- Create a Lambda function named status-api.
- The Lambda function must be deployed within the App VPCs in both EU and AP regions.
- Configure the function to save access logs, including the request time and response code.

Observability

- Enable Performance Insights for the Aurora RDS with a retention period of 7 days.
- Enable VPC Flow Logs for all VPCs, restricted to the allowed type only.
- Enable all types of logs for all EKS clusters, and store them in CloudWatch Logs under: /eks/greentree/{cluster name}/*

- Configure the Lambda functions to save logs in CloudWatch Logs: /lambda/greentree/status/*
- Configure the Plant and Grow Pods to save logs in CloudWatch Logs: /app/greentree/{plant | grow}/*
- Create a CloudWatch Dashboard that includes the following metric graphs:
  - EC2 CPU Utilization
  - EC2 Network Bandwidth
  - Pod CPU Utilization
  - Pod Memory Usage
  - DynamoDB Capacity (WCU/RCU)
  - RDS CPU Utilization

Application

- Build container images for the Plant and Grow applications.
- Deploy the Plant and Grow applications as containers on EKS.
- Deploy the status application as an AWS Lambda function.
- Both the Plant and Grow applications require a config.ini file located in the same path as the binary.
- The Grow binary /growadmin
- Provide a single endpoint over the HTTPS protocol to access all three apps: Plant, Grow and Status.
- The endpoint must not be region-specific and must route traffic to the region with the lowest latency.
- The endpoint address is: https://api.<competitor's base domain>
- All APIs must respond within 1.8 seconds.
- Do not alter the provided binaries.

API	Request	Response	Description
POST /plant	application/json - treeId: Unique ID (String) - treeName: Tree name (String) Sample: {"treeId": "xx11qqxx", "treeName": "bamboo"}	Code 201 - status: planted or failed (String) - hash: Validation value (String) - region: DDB region (String) Sample: {"status": "planted", "hash": "xxyy11", "region": "ap-northeast-2"}	
POST /grow	application/json - treeId: Unique ID (String) - sunlight: Sun light (Boolean) - water: Water (Boolean) Sample: {"treeId": "xx11qqxx", "sunlight": true, "water": false}	Code 200 - treeId: ID or Unknown (String) - sunlight: Updated status (Boolean) - water: Updated status (Boolean) - hash: Validation value (String) - region: DDB region (String) Sample: {"treeId": "xx11qqxx" "sunlight": true, "water": false}	If a tree doesn't exist, it returns Unknown. Hash is a validation code.

GET /status	Query string - treeId Sample: /status?treeId=xx11qqxx	Code 200 - status: Tree status sprout | growing | grown (String) - name: Tree Name (String) - region : Lambda deployed location "ap-northeast-2" or "eu-west-1" (String) Sample: {"status": "sprout", "name": "bamboo", "region": "ap-northeast-2"}	If sunlight and water are False, sprout. One of them is true, growing. All are true, grown. Region is Lambda function Region.

Reference

- https://www.keycloak.org/
- https://argo-cd.readthedocs.io/en/stable/operator-manual/declarative-setup/
- https://aws-controllers-k8s.github.io/community/reference/dynamodb/v1alpha1/table/